

Deep Learning

Paweł Jacyno

Przygotowanie danych

- Przygotowanie Danych do Treningu Pracę rozpocząłem od wczytania pliku `train_clean.csv`, który został wcześniej przygotowany przez innego członka zespołu. Plik ten zawierał już wyczyszczone i zlematyzowane teksty w kolumnie `clean_lemma` oraz etykiety.
- Podział danych: Wczytana tabela została podzielona na zbiór treningowy (80%) i walidacyjny (20%). Taki podział jest kluczowy, aby móc obiektywnie ocenić, jak model radzi sobie z danymi, których wcześniej nie widział.
- Tokenizacja "w locie": Zamiast tworzyć dodatkowe pliki, zaimplementowano własną klasę `Dataset` w PyTorch. Jej zadaniem było dynamiczne przetwarzanie tekstu w momencie podawania go do modelu. Proces ten obejmował:
- Tokenizację przy użyciu gotowego tokenizera `distilbert-base-uncased`, który zamieniał słowa na odpowiadające im liczby (tokeny).
- Ujednolicenie długości każdego tekstu do 128 tokenów poprzez jego skrócenie lub uzupełnienie (padding).
- `DataLoadery`: Na końcu dane zostały umieszczone w obiektach `DataLoader`, które efektywnie zarządzają podawaniem danych do modelu w małych paczkach (batchach) podczas treningu.

Architektura modelu Bi-LSTM

- Architektura Modelu - Dwukierunkowe LSTM Do zadania klasyfikacji tekstu wybrano architekturę opartą na rekurencyjnej sieci neuronowej typu Long Short-Term Memory (LSTM).
- Warstwa Embedding: Pierwsza warstwa, która zamienia jednowymiarowe wektory tokenów na gęste, wielowymiarowe wektory cech (embeddings), ucząc się semantycznych relacji między słowami.
- Dwukierunkowa warstwa LSTM: Stanowi rdzeń modelu. LSTM jest w stanie analizować sekwencje i pamiętać kontekst. Użycie wariantu dwukierunkowego (bidirectional) pozwoliło modelowi na analizę tekstu zarówno od początku do końca, jak i od końca do początku, co znacząco poprawia zrozumienie kontekstu.
- Warstwa Dropout: Zastosowałem ją jako technikę regularyzacji, która losowo "wyłącza" część neuronów podczas treningu, zapobiegając w ten sposób przeuczeniu się modelu (overfittingowi).
- Warstwa Liniowa (Klasyfikator): Ostatnia warstwa, która na podstawie informacji z LSTM podejmuje ostateczną decyzję, klasyfikując artykuł jako prawdziwy (0) lub fałszywy (1).

Proces treningu i ewaluacji

- Proces Treningu i Ewaluacji Trening modelu
- Optymalizacja: Wykorzystano optymalizator Adam oraz funkcję straty BCEWithLogitsLoss, które są standardem w zadaniach klasyfikacji binarnej.
- Akceleracja GPU: Cały proces treningu odbywał się na karcie graficznej (GPU), co skróciło czas obliczeń z godzin do minut.
- Monitorowanie postępów: Po każdej epoce mierzono stratę (loss) i dokładność (accuracy) zarówno na zbiorze treningowym, jak i walidacyjnym. Wyniki były na bieżąco wizualizowane na wykresach, co pozwalało na śledzenie krzywych uczenia.
- Wczesne Zatrzymywanie (Early Stopping): Zaimplementowano mechanizm, który zapisywał stan modelu tylko wtedy, gdy jego wynik na zbiorze walidacyjnym ulegał poprawie. Zapobiegło to przeuczeniu i zapewniło, że ostatecznie zapisany model jest tym o najlepszej generalizacji.

Wyniki i wnioski

Stworzony model oparty na architekturze Bi-LSTM osiągnął na zbiorze walidacyjnym świetny wynik 98% dokładności. Analiza treningu i metryk pokazuje, że to nie tylko wysoka, ale i wiarygodna skuteczność.

Model jest bezstronny. Z precyzją i czułością na poziomie 98% zarówno dla wiadomości prawdziwych, jak i fałszywych, udowodnił, że jest równie dobry w wyłapywaniu dezinformacji, co w przepuszczaniu wiarygodnych newsów. To kluczowe, bo system, który wszystko oznacza jako "fake", jest bezużyteczny.

Wykresy potwierdziły słuszność strategii. Po około 7. epoce model zaczął się przeuczać – stawał się za dobry na danych treningowych kosztem nowych, nieznanych danych. To idealnie pokazuje, że dodanie mechanizmu Early Stopping było strzałem w dziesiątkę. Trening został zatrzymany w idealnym momencie, zapisując model o najlepszej możliwej skuteczności.

Model rzadko przepuszcza fejki. Macierz pomyłek potwierdza, że tylko 161 fałszywych newsów (z ponad 7400) zostało błędnie oznaczonych jako prawdziwe. To najważniejszy błąd, który chciałem zminimalizować, i wynik jest bardzo zadowalający.